

WaxTree — Documentazione tecnica completa

PANORAMICA

WaxTree è una web app per DJ diggers che permette di esplorare artisti e label musicali costruendo un **albero non-distruttivo** di nodi collegati. Ogni nodo è un artista o una label; i nodi sono organizzati in rami (branches) indipendenti e arricchiti con tracce da Discogs, video da YouTube, copertine da MusicBrainz, e link Bandcamp da Serper/Google.

STACK TECNOLOGICO

Layer	Tecnologia
Frontend	HTML/CSS/JS vanilla, <code><script type="module"></code> , nessun build step
Backend auth	Supabase (PostgreSQL + GoTrue)
Edge Function	Supabase Edge Function (Deno/TypeScript)
Discogs	REST API pubblica con Personal Access Token
MusicBrainz	REST API pubblica (cover art)
CoverArtArchive	REST API pubblica (immagini copertine)
Wikipedia	REST API pubblica (bio artisti)
YouTube	YouTube IFrame API (<code>youtube-nocookie.com</code>)
Bandcamp	Bandcamp Autocomplete API + Google via Serper.dev
Library scan	File System Access API (browser-native, Chrome/Edge only)
Metadata audio	<code>music-metadata-browser@2.5.10</code> via <code>esm.sh</code>
Persistenza locale	<code>localStorage</code> (chiave <code>ct2:</code> + <code>SK='ct-v5'</code>)

ARCHITETTURA FRONTEND

Il file unico è `preview.html` (~1780 righe). Non esiste bundler né framework JS.

Pattern di rendering

Il DOM viene completamente ricreato a ogni aggiornamento di stato:

```
modifica stato → rr() → requestAnimationFrame → paint() → buildApp()
```

- `rr()` — debounce: se c'è già un frame pending non ne accoda un altro; salva stato in `localStorage` prima del render
- `paint()` — salva gli scroll position dei container con `data-sc`, svuota `#root`, ricostruisce l'intero DOM, ripristina gli scroll
- `saveSt()` — serializza `st` in `localStorage[SK]` ad ogni render

Costruzione DOM

Nessun framework, helper custom: - `el(tag, attrs, ...children)` — createElement generico - `d(cls, attrs, ...ch)` — shorthand per `<div>` - `s(cls, ...ch)` — shorthand per `<span>` - `b(cls, attrs, ...ch)` — shorthand per `<button>`

STRUTTURA DATI

Stato globale `st`

```
{
  // Persistito in localStorage
  theme: 'dark' | 'light',
  branches: Branch[],
  nodes: Node[],
  selectedId: string | null,      // id del nodo selezionato
  activeBranchId: string,        // id del branch attivo
  chips: string[],               // recent searches bar (max 12)
  likes: { [trackId]: boolean },
  listens: { [trackId]: { badged: boolean } },
  dasAscoltare: Track[],         // "Listen later" queue
  history: HistoryEntry[],       // tracce ascoltate ≥3 sec
  sbPinFirst: boolean,           // sidebar: pinned nodes prima

  // Non persistito (runtime)
  q: string,                     // query search corrente
  results: SearchResult[],       // risultati disambig dropdown
  loading: boolean,
  err: string,
  modal: boolean,               // token modal aperto
  tokenDraft: string,
  bioOpen: { [nodeId]: boolean },
  renameId: string | null,
  renameVal: string,
  tagNodeId: string | null,
```

```

tagVal: string,
filterOpen: boolean,
filterTitle: string,
filterFormat: 'all' | 'digital' | 'vinyl',
filterSort: 'default' | 'chrono' | 'antichrono' | 'az' | 'za' | 'genre',
sbFilterTag: string,
nowPlaying: NowPlaying | null,
daqModal: boolean,
profileOpen: boolean,
likesModal: boolean,
historyModal: boolean,
settingsModal: boolean,
profileModal: boolean,
searchCount: number,           // contatore per gamification
ownedTracks: OwnedTrack[],      // tracce dalla libreria locale
}

```

Branch

```

{
  id: string,    // 'b' + Date.now()
  name: string   // "Branch 1", modificabile
}

```

Node

```

{
  id: string,           // 'n' + Date.now() | 'd1' (demo)
  branchId: string,
  type: 'artist' | 'label',
  discogsId: number,
  name: string,
  parentId: string | null, // null = root del branch
  pinned: boolean,
  tags: string[],
  loaded: boolean,
  loading: boolean,
  error: string | null,
  data: ArtistData | LabelData | null
}

```

ArtistData (da Discogs + Wikipedia + MusicBrainz)

```
{
  id: number,
  name: string,
  bio: string | null,
  imageUrl: string | null,
  aliases: Array<{ id, name, type: 'artist' | 'label' }>,
  highlights: {
    yearRange: string | null,    // "1986-2022"
    names: string[],            // label principali
    labelStr: string | null,
    curiosity: string | null
  },
  correlatedArtists: string[],  // artisti della stessa label principale
  tracks: Track[],
  trackCount: number           // totale su Discogs (non caricati tutti)
}
```

LabelData (da Discogs + Wikipedia + MusicBrainz)

```
{
  id: number,
  name: string,
  bio: string | null,
  imageUrl: string | null,
  sublabels: Array<{ id, name, type }>,
  parentLabel: { id, name, type } | null,
  country: string | null,
  highlights: {
    yearRange: string | null,
    names: string[],           // artisti della label
    artistStr: string | null
  },
  tracks: Track[],
  trackCount: number
}
```

Track

```
{
  id: string,                // "${releaseId}-${position}"
  title: string,
  duration: string,          // "5:47"
}
```

```

year: number | null,
label: string,
labelId: number | null,
genre: string | null, // max 2 stili Discogs
thumbUrl: string | null,
bpm: number | null, // estratto da note release/traccia
album: string, // titolo release
digital: boolean, // da formats[].name o descriptions
discogsUrl: string,
videoId: string | null, // YouTube video ID (11 chars)
catno: string,
fromLabel: boolean, // true = nodo è una label
exploreId: number | null, // id per nodo figlio da "Explore"
exploreName: string | null,
exploreType: 'artist' | 'label' | null,
exploreLabel: string | null // testo pulsante "Explore"
}

```

BcCacheMap (in-memory, non persistito)

```

bcCacheMap: {
  [nodeId]: {
    tracks: Array<{ url, title, artist, album, released, thumb }>,
    loading: boolean,
    err: string | null
  }
}

```

OwnedTrack (libreria locale)

```

{
  titleNorm: string, // titolo normalizzato (lowercase, alphanum only)
  artistNorm: string,
  filename: string
}

```

HistoryEntry

```

{
  id: string, // trackId
  title: string,
  artistName: string,

```

```
ts: number // Date.now()
}
```

FUNZIONALITÀ

1. Autenticazione (Supabase Auth)

- All'avvio (`sb.auth.getSession()`) verifica la sessione; se assente, redirect a `login.html`
- La sessione Supabase viene salvata in `wtSession`
- User metadata Supabase: `search_count` , `owned_tracks` , `library_scanned_at` , `library_track_count`
- **Change password:** inline nel modal Settings, chiama `sb.auth.updateUser({ password })`
- **Sign out:** `sb.auth.signOut()` + redirect a `login.html`

2. Token Discogs

- Salvato in `localStorage['discogs_token']` (mai nel codice, mai in chat)
- Inserito tramite modale  nell'header
- Usato in ogni richiesta Discogs come `Authorization: Discogs token=...`
- Rate limiting: max 55 req/min con counter `rqN` / `rqW` ; se superato attende 62 secondi

3. Ricerca artisti / label

Flusso: 1. Utente digita nella search bar e preme Enter / “Search” 2. `doSearch()` → `searchDiscogs(q)` → `GET /database/search?q=...&per_page=25` 3. Filtra solo `type=artist` o `type=label` 4. Se 1 risultato: `pickResult()` diretto → `addNode()` 5. Se più risultati: dropdown disambig (max 14) con thumbnail, tipo e nome 6. Ogni ricerca incrementa `st.searchCount` → trigger gamification

Cache: `lsGet('s:'+query)` / `lsSet('s:'+query, results)` — TTL 24h

4. Grafo nodi (Branches + Nodes)

- **Branch:** tab nella sidebar; ogni nodo appartiene a un branch
- `addNode()` — crea un nodo, imposta `loading:true` , chiama `fetchArtistData` o `fetchLabelData` , aggiorna `node.data` quando la promise risolve, poi avvia `fetchBandcamp`
- **Nodo figlio:** quando si clicca “Explore artist/label” su una traccia, il nuovo nodo diventa figlio del nodo corrente (`parentId`)
- **Breadcrumb:** `ancestry(nodeId)` risale la catena di `parentId` e costruisce il breadcrumb nel content panel
- **Selezione:** `selectNode(id)` imposta `selectedId` , resetta i filtri, triggera re-render
- **Pin:** `togglePin(nodeId)` — se `sbPinFirst=true` i nodi pinnati vengono mostrati per primi
- **Tag:** aggiunta/rimozione inline nel sidebar item; dropdown di filtro per tag nel sidebar filter bar

- **Drag & drop:** ogni nodo sidebar è `draggable` ; i tab branch accettano drop → `moveNodeToBranch(nodeId, branchId)`
- **Rename branch:** doppio click sul tab → inline input
- **Rimozione branch:** elimina branch e tutti i suoi nodi; almeno 1 branch deve rimanere

5. Fetch dati artista (`fetchArtistData`)

1. `GET /artists/:id` — nome, bio, immagine, aliases
2. `GET /artists/:id/releases?per_page=100&sort=year` — lista release
3. Filtra per `role==='Main'` ; se nessuna, usa tutte
4. Per ogni release (max 200 tracce):
 - Se `type==='master'` e ha `main_release` → `GET /releases/:main_release`
 - Se `type==='master'` senza `main_release` → `GET /masters/:id`
 - Altrimenti → `GET /releases/:id`
5. `buildTrackEntries()` : estrae tracklist (max 4 tracce per release), formato, BPM da note, video YouTube per match titolo
6. Se la release non ha copertina: `fetchMbCoverUrl()` su MusicBrainz
7. Se bio < 60 chars o assente: `fetchWikipediaData()` per bio e immagine Wikipedia
8. Artisti correlati: trova la label più frequente tra le tracce → `GET /labels/:topLabelId/releases?per_page=30` → top 10 artisti diversi dall'artista corrente
9. Deduplication: `dedup()` — identifica duplicati per titolo normalizzato + bucket durata (30s)
10. Cache: `lsSet('a7:'+discogsId, data)` — saltata se `tracks.length===0`

6. Fetch dati label (`fetchLabelData`)

1. `GET /labels/:id` — nome, bio, immagine, sublabels, parent_label, country
2. `GET /labels/:id/releases?per_page=100&sort=year` — release della label
3. Per ogni release: `GET /releases/:id` → `buildTrackEntries()` ; aggiunge link “Explore artist” sull'artista principale della release
4. Wikipedia + MusicBrainz come per artisti
5. Cache: `lsSet('l7:'+discogsId, data)`

7. Tracce e filtri

Pannello content per nodo caricato: - Mostra le tracce in pagine da **50** (`tracksPageMap[nodeId]`) - Filtro titolo (substring case-insensitive) - Filtro formato: tutti / digital / vinyl - Ordinamento: default / cronologico / antichronologico / A→Z / Z→A / per genere - **Float a top automatico:** tracce con video YouTube valido (non nella `invalidYtIds`) flotano in cima; tra queste, le digital hanno precedenza -  `Filter` • mostra punto se filtro attivo

8. Track card

Per ogni traccia viene renderizzato un card con:

Elemento	Descrizione
► Play	Apri YouTube mini-player; se nessun video → fallback search YouTube
🖼 Copertina	t-art (img) o placeholder 🎵
Titolo + meta	Anno · Label · Genere
“You own it”	Badge verde se la traccia è nella libreria locale (match fuzzy)
✓ played	Badge se ascoltata ≥3 secondi nel player
🔍/	Aggiungi/rimuovi da “Listen later”
♡/♥	Like/unlike
BPM	Se estratto dalle note Discogs
Durata	
Explore label	Cerca la label (per labelId o per nome)
Explore artist/alias	Aggiunge nodo figlio
Bandcamp ↗	Cerca URL diretto (vedi §10)
Beatport ↗	Solo su tracce digital, link search Beatport
Discogs ↗	Link diretto alla release

9. YouTube mini-player

- **Posizione:** fixed bottom-right, draggable (posizione salvata in `localStorage['wt-player-pos']`)
- **IFrame API:** caricata lazy da `youtube.com/iframe_api`; usa `youtube-nocookie.com`
- **Badge “✓ played”:** triggerato dopo 3 secondi di ascolto continuo (`ytBadgeTimerId`) O dopo 2+ seek (`ytSkips`)
- **Pre-validazione video:** `prevalidateYt(id)` controlla se il thumbnail `mqdefault.jpg` ha larghezza > 120px; se no, aggiunge l’ID a `invalidYtIds` e il pulsante ► diventa search YouTube
- **Errori IFrame:** codici 100/101/150/2/5 → aggiunge a `invalidYtIds`, mostra fallback con link search
- **History:** aggiornata quando `tryBadge()` succede; salvata in `st.history`
- `trackId`, `videoId`, `title`, `artistName` in `st.nowPlaying`

10. Bandcamp URL discovery

Artist-level (pre-fetch): Quando un nodo viene selezionato, `fetchBandcamp(nodeId, artistName)` chiama l’Edge Function con `{ artist: artistName }` e salva i risultati in `bcCacheMap[nodeId]`.

Click-level (on demand): Al click su “Bandcamp ↗”: 1. Apre subito `about:blank` (evita popup blocker) 2. Chiama Edge Function con `{ artist, title, release: track.album }` 3. Logga `[BC debug]` in console 4. Se trova un URL → naviga il tab aperto; altrimenti usa fallback search Bandcamp

`findBcMatch(parentNodeId, title)` : Fuzzy match tra titolo richiesto e tracce in `bcCacheMap` : 1. Exact match → URL diretto (badge `.direct`) 2. Substring containment → score = ratio lunghezze 3. Word overlap → score = overlap / max(words); soglia 0.5

Edge Function `bc-search` (Deno): Strategie in ordine di priorità: 1. Bandcamp Autocomplete API `autocomplete_elastic?q=artist+keyword` → `item_url` diretto (album/track) 2. Bandcamp Autocomplete API con solo `artist` 3. Google via Serper.dev `site:bandcamp.com artist+keyword` (richiede `SERPER_API_KEY` nei secrets Supabase) 4. Google via Serper.dev `site:bandcamp.com artist`

`isBcArtistUrl(url)` : accetta qualsiasi URL con `.bandcamp.com` (sottodomain artista), esclude `bandcamp.com/search`.

11. Libreria locale (Local Library Scanner)

Disponibile su Chrome/Edge (richiede File System Access API).

Flusso in 2 fasi: 1. **Phase 1** — `collectAudioFiles(dirHandle)` : ricorsione nella directory, raccoglie solo gli handle dei file con estensione audio (`.mp3 .flac .aiff .aif .wav .m4a .ogg`). Solo lettura nome file, nessuna apertura. 2. **Phase 2** — `extractAllMetadata(handles)` : elabora in batch da 50 in parallelo (`BATCH_SIZE=50`). Per ogni file: - Prova a leggere i tag ID3/FLAC/etc. con `music-metadata-browser` (`title`, `artist`) - Fallback: `parseFilename()` — split su `-` o `_`, rimuove track number prefix

Risultato: array di `OwnedTrack[]` salvato in `st.ownedTracks` e sincronizzato su Supabase (`sb.auth.updateUser({ data: { owned_tracks, library_scanned_at, library_track_count } })`).

Badge “You own it”: `isOwned(title, artist)` confronta titolo e artista normalizzati (solo alfanumerico lowercase) contro `st.ownedTracks`. Match fuzzy: substring bidirezionale su titolo, poi su artista.

`normalizeStr(str)` : lowercase → rimuove non-alfanumerico → rimuove stopword musicali (`original mix`, `remix`, `edit`, `remaster`, `ep`, `lp`, `feat`, `ft`).

12. Gamification (15 livelli)

Ogni ricerca completata incrementa `st.searchCount` e chiama `sb.auth.updateUser({ data: { search_count } })`.

`getLevelFromCount(count)` — trova il livello corrente. `getProgressToNext(count)` — percentuale progressione al livello successivo.

Al level-up: toast animato `#level-toast` visibile 4 secondi.

Livelli da 1 (“First Seed”, 0 ricerche) a 15 (“The Root”, 10001+). Livello 16 (“The Mycelium”) riservato, non renderizzato.

13. Chips (recent searches)

Barra sotto la search: ultimi 12 artisti/label cercati. - Click su chip → se il nodo esiste nella sidebar selezionarlo; altrimenti `doSearch()` - `x` per rimuovere - Nuovo nodo aggiunto → nome aggiunto in testa

14. “Listen later” (Da ascoltare)

- Bottone  su ogni traccia → aggiunge a `st.dasAscoltare`
-  nel header con badge contatore
- Modal con lista, play diretto, rimozione singola, “Clear all”

15. Likes

- /♥ su ogni traccia → toggle `st.likes[track.id]`
- Modal “My Likes” nel menu Profile → aggrega le tracce piaciute da tutti i nodi caricati

16. History

- Aggiornata da `tryBadge()` quando una traccia è stata ascoltata ≥ 3 secondi
- Modal “History” con timestamp formattato
- Persistita in `st.history[]` in localStorage

17. Sezione Bandcamp nel content panel

Mostra i risultati del `bcCacheMap[nodeId]` : - Loading spinner durante fetch - Se `tracks.length > 0` : lista con thumbnail, titolo, meta, link “Listen ↗” - Se `tracks.length === 0` : messaggio “No results on Bandcamp” o errore

18. Artisti correlati

Per nodi artista: sezione “Related artists” con i nomi degli artisti sulla stessa label principale. Click su un chip → `doSearch(name)` .

19. Aliases e sublabel

- **Artista**: pills cliccabili per ogni alias (max 6) → `addNode('artist', alias.id, ...)`
- **Label**: pills per parent label (↑) e sublabels (max 6) → `addNode('label', ...)`

20. Impostazioni (Settings modal)

- **Account**: email (read-only), cambio password inline
- **Local Library**: link/rescan cartella, data e conteggio ultimo scan
- **Data**: clear history, reset posizione mini-player, reset totale dati (localStorage)

21. Dark/Light theme

Toggle  nell'header. Applica `data-theme="dark"|"light"` su `<html>` . Le variabili CSS `--bg` , `--accent` , ecc. cambiano di conseguenza. Persistito in `st.theme` .

CACHE LOCALSTORAGE

Chiave	Contenuto	TTL
ct2:a7:{discogsId}	ArtistData completo	24h
ct2:l7:{discogsId}	LabelData completo	24h
ct2:s:{query}	Risultati search Discogs	24h
ct2:mb:{artist}:{title}	URL copertina MusicBrainz	24h
ct-v5	Stato app (branches, nodes, likes, ecc.)	Permanente
discogs_token	Token API Discogs	Permanente
wt-player-pos	Posizione mini-player {x, y}	Permanente

API ESTERNE

API	Endpoint base	Auth	Limite
Discogs	https://api.discogs.com	Personal token header	55 req/min
MusicBrainz	https://musicbrainz.org/ws/2	User-Agent header	~1 req/sec
CoverArtArchive	https://coverartarchive.org	nessuna	—
Wikipedia	https://en.wikipedia.org/api/rest_v1	User-Agent header	ragionevole
YouTube IFrame	youtube-nocookie.com	nessuna (embed pubblico)	—
Bandcamp Autocomplete	https://bandcamp.com/api/bcsearch_public_api/1/autocomplete_elastic	nessuna	non documentato
Serper.dev (Google)	https://google.serper.dev/search	X-API-KEY header	2500 free
Supabase Auth	https://asmnqlqvlpwcaaughuu.supabase.co	JWT session	—
Supabase Edge Function bc-search	via sb.functions.invoke()	JWT session	—

FILE DI PROGETTO

```
crate-tree/
├─ preview.html          # App intera (~1780 righe)
├─ login.html            # Pagina login (Supabase)
├─ supabase/
│   └─ functions/
│       └─ bc-search/
│           └─ index.ts  # Edge Function Bandcamp search
└─ PLATFORM_DOCS.md      # Questo documento
```